硬件初始化与 LVGL 绑定

硬件初始化与 LVGL 绑定
1. 课程目标
2. 核心源码分析
2.1 显示系统初始化 (`display_init`)
2.2 LVGL 图形库初始化 (`lvgl_init`)
2.3 核心回调：`disp_drv_flush_cb`
3. 触摸屏驱动 (`touch_screen` 类)
4. 总结

1. 课程目标

在上一节中，我们看到了 `main.py` 调用了 `start()`。本节课我们将深入 `ybMain/main.py`，剖析系统是如何"点亮"屏幕，并将 LVGL 图形库与 K230 的底层硬件（显示屏、触摸屏）绑定在一起的。

2. 核心源码分析

文件路径: `src/canmv/port/builtin_py/ybMain/main.py`

2.1 显示系统初始化 (`display_init`)

K230 的显示系统比普通单片机复杂，涉及到 **PipeLine (视频通路)** 的概念 (使用Pipeline是为了方便进行多路的AI视觉分析任务)。

```
def display_init():
    global pl, config
    try:
        # ...
        display_mode = "lcd"
        display_size = [DISPLAY_WIDTH, 480]
        rgb888p_size = [640, 480]

        # 初始化 PipeLine，这是 K230 多媒体框架的核心
        # osd_layer_num=4 表示预留了 4 个图层用于屏幕显示 (On-Screen Display)
        pl = PipeLine(rgb888p_size=rgb888p_size, display_size=display_size,
display_mode=display_mode, osd_layer_num=4)

        # 创建显示通道
        pl.create(...)
    except Exception as e:
        debug_print("display_init", e)
```

关键点:

- **PipeLine**: 这是一个专门处理图像数据的硬件通道。GUI 并不是直接写显存，而是作为一个"图层"叠加在视频输出上。这使得我们可以轻松地在 GUI 背景中显示摄像头画面。

2.2 LVGL 图形库初始化 (`lvgl_init`)

LVGL 是一个纯软件的图形库，它不知道如何画点。我们需要提供一个“回调函数”告诉它如何将像素推送到屏幕。

```
def lvgl_init():
    # 1. 初始化 LVGL 库
    lv.init()

    # 2. 创建显示驱动
    disp_drv = lv.disp_create(DISPLAY_WIDTH, DISPLAY_HEIGHT)

    # 3. 设置刷新回调函数 (最重要的部分!)
    disp_drv.set_flush_cb(disp_drv_flush_cb)

    # 4. 分配双缓冲区 (Double Buffering)
    disp_img1 = image.Image(DISPLAY_WIDTH, DISPLAY_HEIGHT, image.BGRA8888)
    disp_img2 = image.Image(DISPLAY_WIDTH, DISPLAY_HEIGHT, image.BGRA8888)
    disp_drv.set_draw_buffers(disp_img1.bytearray(), disp_img2.bytearray(), ...)

    # 5. 初始化输入设备 (触摸屏)
    global_touch_dev = touch_screen()
```

2.3 核心回调：`disp_drv_flush_cb`

这是连接软件 (LVGL) 和硬件 (Display) 的桥梁。

```
def disp_drv_flush_cb(disp_drv, area, color):
    # 当 LVGL 渲染完一部分画面后，会调用这个函数
    # color: 指向渲染好的像素数据

    if disp_drv.flush_is_last() == True:
        # 如果这是当前帧的最后一块数据，就调用 Display.show_image 上屏
        if disp_img1.virtaddr() == ...:
            Display.show_image(disp_img1)
        else:
            Display.show_image(disp_img2)

    # 告诉 LVGL 刷新完成，可以准备下一帧了
    disp_drv.flush_ready()
```

工作原理解析:

1. LVGL 在内存中画好界面。
2. 调用 `disp_drv_flush_cb`，传入画好的数据。
3. 我们调用 K230 的 `Display.show_image()` 将这块内存的数据发送给显示控制器。
4. 使用 **双缓冲** (`disp_img1`, `disp_img2`) 可以避免画面撕裂，保证 UI 流畅。

3. 触摸屏驱动 (`touch_screen` 类)

为了让屏幕能响应点击，我们需要将物理触摸坐标转换为 LVGL 理解的坐标。

```
class touch_screen():
    def __init__(self):
        # 创建 LVGL 输入驱动
```

```
        self.indev_drv = lv.indev_create()
        self.indev_drv.set_type(lv.INDEV_TYPE.POINTER)
        self.indev_drv.set_read_cb(self.callback) # 注册回调

    def callback(self, driver, data):
        # 读取物理触摸数据
        tp = self.touch.read(1)
        if len(tp):
            # 将物理坐标赋值给 LVGL 数据结构
            data.point = lv.point_t({'x': tp[0].x, 'y': tp[0].y})
            data.state = lv.INDEV_STATE.PRESSED
        else:
            data.state = lv.INDEV_STATE.RELEASED
```

4. 总结

本节课中，我们了解了 GUI 系统的“地基”：

1. **PipeLine** 负责打通视频通路。
2. **LVGL** 负责生成图形数据。
3. **flush_cb** 负责将图形数据搬运到视频通路上。
4. **touch_screen** 负责将手指动作翻译成点击事件。

下一节，我们来讲解一下GUI的核心部分：**AppManager**，看看它是如何管理那些花花绿绿的应用图标的。